

JavaScript 기본 개념 학습 가이드

변수, 자료형, 배열/객체, 비동기 처리, 모듈까지 한 문서로 정리

이 문서의 목표 JavaScript를 처음 공부하거나 React에서 FastAPI를 호출해야 하는 상황을 기준으로, 문법을 외우는 수준이 아니라 코드가 어떻게 읽히고 왜 그렇게 쓰이는지 이해하도록 구성했습니다. 각 개념은 정의, 사용 이유, 기본 예제, 주의점을 함께 다룹니다.

읽는 순서

- 기본 문법에서 값을 만들고 흐름을 제어하는 방법을 먼저 익힙니다.
- 배열과 객체 다루기에서 실제 데이터 변환 패턴을 배웁니다.
- 비동기 처리에서 서버와 통신하는 코드의 흐름을 이해합니다.
- 모듈에서 React 프로젝트처럼 파일을 나누고 연결하는 방법을 익힙니다.

0. 전체 지도

JavaScript는 웹 브라우저와 Node.js 환경에서 동작하는 프로그래밍 언어입니다. React 컴포넌트도 JavaScript를 바탕으로 작성되며, 서버와 통신할 때도 JavaScript 코드가 HTTP 요청을 만들고 응답을 처리합니다.

큰 범주	무엇을 배우는가
값	string, number, boolean, null, undefined처럼 프로그램이 다루는 데이터입니다.
그릇	let, const 변수는 값을 이름으로 저장합니다. 배열과 객체는 여러 값을 묶습니다.
흐름	if, switch, for 같은 문법은 어떤 코드를 언제 실행할지 결정합니다.
동작	함수와 화살표 함수는 반복해서 사용할 작업을 이름 붙인 것입니다.
통신	Promise, async/await, fetch는 서버 응답처럼 나중에 도착하는 값을 다룹니다.
파일 분리	import/export는 코드를 여러 파일로 나누어 재사용하게 해 줍니다.

핵심 독해법

JavaScript 코드는 대체로 '값을 만든다 -> 조건에 따라 고른다 -> 배열/객체를 변환한다 -> 함수로 묶는다 -> 필요하면 서버와 비동기로 통신한다'는 흐름으로 읽으면 훨씬 쉬워집니다.

1. 기본 문법

1.1 변수 선언: let, const

변수는 값을 이름으로 저장해 두는 공간입니다. JavaScript에서 현대적인 변수 선언은 주로 let과 const를 사용합니다.

왜 필요한가: 같은 값을 여러 번 쓰거나, 중간 계산 결과에 의미 있는 이름을 붙이면 코드가 읽기 쉬워집니다.

- let은 나중에 다른 값을 다시 대입할 수 있습니다.
- const는 같은 변수 이름에 다시 대입할 수 없습니다. 단, 객체나 배열 내부 값까지 완전히 열리는 것은 아닙니다.
- var도 있지만 스코프 규칙이 헷갈리기 쉬워 새 코드에서는 보통 let/const를 씁니다.
- 가능하면 const를 먼저 쓰고, 값이 바뀌어야 할 때만 let을 쓰는 습관이 좋습니다.

```
const userName = "Yoonji";
let count = 0;

count = count + 1; // let은 재할당 가능

const user = { name: "Yoonji", age: 20 };
user.age = 21; // const여도 객체 내부 속성 변경은 가능

// user = { name: "Mina" }; // 에러: const 변수 자체를 다시 대입할 수 없음
```

주의

const는 '값이 절대 변하지 않는다'가 아니라 '이 변수 이름이 다른 값으로 다시 연결되지 않는다'에 가깝습니다.

1.2 자료형: string, number, boolean, null, undefined

자료형은 값의 종류입니다. JavaScript는 변수에 자료형을 고정하지 않는 동적 타입 언어라서, 한 변수에 문자열을 넣었다가 나중에 숫자를 넣는 것도 문법상 가능합니다.

- string은 문자 데이터입니다. 작은따옴표, 큰따옴표, 백틱으로 만들 수 있습니다.
- number는 정수와 소수를 모두 포함합니다. 10, 3.14, -5가 모두 number입니다.
- boolean은 true 또는 false 두 값만 가집니다.
- null은 개발자가 '값이 비어 있음'을 의도적으로 넣은 값입니다.
- undefined는 값이 아직 할당되지 않았거나 존재하지 않는 속성을 읽을 때 주로 나옵니다.

```
const title = "JavaScript";
const price = 12000;
const isLoggedIn = true;
const selectedUser = null;

let message;
console.log(message); // undefined

console.log(typeof title); // "string"
console.log(typeof price); // "number"
console.log(typeof isLoggedIn); // "boolean"
```

자료형	대표 사용 상황
string	화면에 보여줄 글자, 이름, 이메일, URL 같은 텍스트에 사용합니다.
number	가격, 수량, 점수, 좌표, 페이지 번호처럼 계산 가능한 값에 사용합니다.
boolean	로그인 여부, 체크 여부, 성공/실패처럼 둘 중 하나를 표현합니다.
null	선택된 사용자가 아직 없음처럼 '비어 있음'을 의도적으로 표현합니다.
undefined	아직 값이 없거나 객체에 그런 속성이 없을 때 나타납니다.

Truthy / Falsy

조건문에서 false처럼 취급되는 값은 false, 0, 빈 문자열(""), null, undefined, NaN입니다. 그 외 대부분의 값은 true처럼 취급됩니다. 그래도 명확한 비교가 필요할 때는 user !== null처럼 직접 비교하는 편이 안전합니다.

1.3 배열: Array

배열은 여러 값을 순서대로 담은 자료구조입니다. 각 값은 0부터 시작하는 인덱스로 접근합니다.

왜 필요한가: 서버에서 받은 게시글 목록, 사용자 목록, 장바구니 상품처럼 '여러 개의 같은 성격의 데이터'를 다룰 때 가장 많이 씁니다.

- 첫 번째 요소의 인덱스는 0입니다.
- length 속성은 배열의 길이를 알려 줍니다.

- 배열 안에는 문자열, 숫자, 객체 등 어떤 값이든 들어갈 수 있습니다.
- React에서는 배열을 map으로 컴포넌트 목록으로 바꾸는 패턴이 매우 자주 등장합니다.

```
const fruits = ["apple", "banana", "orange"];

console.log(fruits[0]);    // "apple"
console.log(fruits.length); // 3

const users = [
  { id: 1, name: "Ada" },
  { id: 2, name: "Linus" },
];
```

1.4 객체: Object

객체는 관련 있는 값을 key-value 쌍으로 묶는 자료구조입니다. key는 이름표, value는 실제 값입니다.

왜 필요한가: 사용자 한 명, 게시글 하나, 서버 응답 하나처럼 여러 속성을 가진 하나의 대상을 표현하기 좋습니다.

- 객체 속성은 점 표기법(obj.name) 또는 대괄호 표기법(obj['name'])으로 읽습니다.
- 객체 값으로 함수가 들어가면 메서드라고 부릅니다.
- 중첩 객체를 사용하면 더 복잡한 데이터 구조도 표현할 수 있습니다.

```
const user = {
  id: 1,
  name: "Yoonji",
  isAdmin: false,
  profile: {
    city: "Seoul",
  },
};

console.log(user.name);    // "Yoonji"
console.log(user.profile.city); // "Seoul"
```

1.5 조건문: if, else, switch

조건문은 조건의 참/거짓에 따라 다른 코드를 실행합니다.

왜 필요한가: 로그인했을 때와 안 했을 때, 성공 응답과 실패 응답, 점수 구간에 따른 메시지처럼 상황별 분기가 필요합니다.

- if는 가장 기본적인 조건문입니다.
- else if는 여러 조건을 순서대로 검사할 때 씁니다.
- else는 위 조건이 모두 false일 때 실행됩니다.
- switch는 하나의 값이 여러 경우 중 어디에 해당하는지 비교할 때 읽기 좋습니다.
- 비교할 때는 가능하면 == 대신 ===를 사용합니다. ===는 값과 자료형을 함께 비교합니다.

```
const score = 87;

if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else {
```

```

    console.log("C 이하");
  }

  const role = "admin";

  switch (role) {
    case "admin":
      console.log("관리자 화면");
      break;
    case "user":
      console.log("사용자 화면");
      break;
    default:
      console.log("게스트 화면");
  }

```

주의 switch에서 break를 빼면 다음 case까지 이어서 실행될 수 있습니다. 의도한 경우가 아니라면 각 case 끝에 break를 넣습니다.

1.6 반복문: for, for...of, map

반복문은 비슷한 작업을 여러 번 실행합니다. 배열의 각 요소를 처리할 때 특히 자주 씁니다.

- for는 반복 횟수와 인덱스를 직접 제어해야 할 때 씁니다.
- for...of는 배열의 값을 하나씩 꺼내 읽을 때 간결합니다.
- map은 배열의 각 요소를 다른 값으로 바꾸어 새 배열을 만들 때 씁니다.
- React 화면 렌더링에서는 배열.map(...) 패턴이 매우 중요합니다.

```

const numbers = [1, 2, 3];

for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}

for (const number of numbers) {
  console.log(number);
}

const doubled = numbers.map((number) => number * 2);
console.log(doubled); // [2, 4, 6]

```

반복 방식	추천 상황
for	인덱스가 꼭 필요하거나 시작/끝/증가 규칙을 세밀하게 제어할 때
for...of	배열 값을 순서대로 읽기만 하면 될 때
map	원본 배열을 바탕으로 새 배열을 만들 때

1.7 함수 선언

함수는 특정 작업을 수행하는 코드 묶음입니다. 입력값을 매개변수로 받고, 결과값을 return으로 돌려줄 수 있습니다.

왜 필요한가: 같은 작업을 반복해서 쓰거나, 복잡한 로직에 이름을 붙여 읽기 쉽게 만들기 위해 씁니다.

- function 키워드로 선언할 수 있습니다.
- 괄호 안의 이름은 매개변수(parameter)입니다.
- 함수를 호출할 때 넣는 실제 값은 인자(argument)입니다.
- 함수 내부에서 만든 변수는 보통 함수 밖에서 직접 사용할 수 없습니다.

```
function add(a, b) {
  return a + b;
}

const result = add(2, 3);
console.log(result); // 5

function greet(name) {
  console.log(`안녕하세요, ${name}님`);
}

greet("Yoonji");
```

1.8 화살표 함수: () => {}

화살표 함수는 함수를 더 짧게 쓰는 문법입니다. React 코드에서 콜백 함수, 이벤트 핸들러, 배열 메서드와 함께 매우 자주 등장합니다.

- 매개변수가 하나면 괄호를 생략할 수 있지만, 처음에는 괄호를 쓰는 편이 읽기 쉽습니다.
- 본문이 한 줄 표현식이면 중괄호와 return을 생략할 수 있습니다.
- 본문에 여러 문장이 있으면 중괄호를 쓰고 필요한 경우 return을 직접 써야 합니다.
- 화살표 함수는 자신만의 this를 만들지 않습니다. 초급 단계에서는 주로 콜백 함수로 이해하면 충분합니다.

```
const add = (a, b) => {
  return a + b;
};

const multiply = (a, b) => a * b;

const names = ["Ada", "Linus"];
const messages = names.map((name) => `Hello, ${name}`);
```

주의

중괄호를 쓰면 자동 return이 되지 않습니다. `const double = (n) => { n * 2 }`는 `undefined`를 반환합니다.

1.9 return

return은 함수의 결과값을 함수 밖으로 돌려주는 문법입니다. return이 실행되면 함수는 즉시 종료됩니다.

- return 뒤의 값이 함수 호출 결과가 됩니다.
- return이 없으면 함수는 `undefined`를 반환합니다.
- 조건에 따라 일찍 return하면 중첩 if를 줄일 수 있습니다.
- 화살표 함수의 짧은 표현식 본문에서는 return을 생략할 수 있습니다.

```
function getGrade(score) {
  if (score < 0 || score > 100) {
    return "잘못된 점수";
  }

  if (score >= 90) {
    return "A";
  }

  return "B 이하";
}

console.log(getGrade(95)); // "A"
```

1.10 템플릿 리터럴: `\${}`

템플릿 리터럴은 백틱(`)으로 문자열을 만들고, `\${}` 안에 JavaScript 표현식을 끼워 넣는 문법입니다.

왜 필요한가: 문자열과 변수를 연결할 때 + 연산자보다 읽기 쉽고, 여러 줄 문자열도 자연스럽게 만들 수 있습니다.

- 문자열은 백틱으로 감쌉니다.
- `\${}` 안에는 변수, 계산식, 함수 호출 같은 표현식을 넣을 수 있습니다.
- React에서 API URL, 사용자 메시지, 조건부 텍스트를 만들 때 자주 씁니다.

```
const name = "Yoonji";
const count = 3;

const message = `${name}님, 새 알림이 ${count}개 있습니다.`;
console.log(message);

const userId = 10;
const url = `/api/users/${userId}`;
```

2. 배열 / 객체 다루기

실제 프론트엔드 코드에서 데이터는 대부분 배열과 객체 형태로 움직입니다. 서버는 객체 하나를 보내기도 하고, 객체가 여러 개 들어 있는 배열을 보내기도 합니다. 그래서 배열 메서드와 구조분해, spread/rest 문법을 익히면 React 코드가 훨씬 잘 읽힙니다.

2.1 배열 메서드 한눈에 보기

메서드	핵심 역할
map	각 요소를 변환해서 같은 길이의 새 배열을 만듭니다. React 목록 렌더링에 자주 씁니다.
filter	조건을 만족하는 요소만 남긴 새 배열을 만듭니다.
find	조건을 만족하는 첫 번째 요소 하나를 반환합니다. 없으면 undefined입니다.
reduce	배열 전체를 하나의 값으로 누적합니다. 합계, 그룹화, 객체 변환에 씁니다.
forEach	각 요소마다 작업을 실행합니다. 새 배열을 반환하지 않고 undefined를 반환합니다.

2.2 map

map은 배열의 각 요소를 변환해서 새 배열을 반환합니다. 원본 배열의 길이와 결과 배열의 길이는 같습니다.

- 콜백 함수의 return 값이 새 배열의 각 요소가 됩니다.
- 원본 배열을 직접 바꾸는 목적보다는 새 값을 만드는 목적에 맞습니다.
- React에서는 데이터 배열을 JSX 배열로 바꿀 때 사용합니다.

```
const users = [
  { id: 1, name: "Ada" },
  { id: 2, name: "Linus" },
];

const names = users.map((user) => user.name);
console.log(names); // ["Ada", "Linus"]

// React 예시
// users.map((user) => <li key={user.id}>{user.name}</li>)
```

2.3 filter

filter는 조건을 통과하는 요소만 남겨 새 배열을 반환합니다.

- 콜백 함수가 true를 반환한 요소만 결과 배열에 들어갑니다.
- 결과 배열의 길이는 원본보다 작거나 같을 수 있습니다.
- 검색, 카테고리 필터, 완료/미완료 목록 분리 등에 자주 씁니다.

```
const todos = [
  { id: 1, title: "JS 공부", done: true },
  { id: 2, title: "API 연결", done: false },
];
```



```
const unfinished = todos.filter(todo => todo.done === false);
console.log(unfinished); // [{ id: 2, title: "API 연결", done: false }]
```

2.4 find

find는 조건을 만족하는 첫 번째 요소를 반환합니다. 조건을 만족하는 요소가 없으면 undefined를 반환합니다.

- 하나만 찾으면 되는 상황에 적합합니다.
- id로 특정 객체를 찾을 때 자주 씁니다.
- 반환값이 undefined일 수 있으므로 사용 전에 확인하는 습관이 좋습니다.

```
const users = [
  { id: 1, name: "Ada" },
  { id: 2, name: "Linus" },
];

const user = users.find((item) => item.id === 2);

if (user) {
  console.log(user.name); // "Linus"
}
```

2.5 reduce

reduce는 배열을 하나의 결과값으로 누적합니다. 숫자 합계뿐 아니라 객체 만들기, 그룹화, 최대값 찾기에도 사용할 수 있습니다.

- 첫 번째 인자는 누적값(accumulator), 두 번째 인자는 현재 요소(current item)입니다.
- 두 번째 인자로 초기값을 넣는 것이 안전합니다.
- 처음에는 map/filter/find보다 어렵게 느껴질 수 있으므로, '배열을 하나의 값으로 접는다'고 이해하면 좋습니다.

```
const prices = [1000, 2500, 3000];

const total = prices.reduce((sum, price) => {
  return sum + price;
}, 0);

console.log(total); // 6500

const users = [
  { id: 1, name: "Ada" },
  { id: 2, name: "Linus" },
];

const usersById = users.reduce((acc, user) => {
  acc[user.id] = user;
  return acc;
}, {});

console.log(usersById[1].name); // "Ada"
```

주의

reduce에서 초기값을 생략하면 빈 배열에서 에러가 나거나 의도와 다른 자료형으로 시작할 수 있습니다. 초급 단계에서는 항상 초기값을 넣는 습관을 추천합니다.

2.6 forEach

forEach는 배열의 각 요소마다 어떤 작업을 실행합니다. 하지만 새 배열을 만들어 반환하지는 않습니다.

- console.log, 외부 배열에 push, 통계 기록처럼 부수 효과가 목적일 때 사용합니다.
- 반환값은 undefined입니다.
- 데이터를 변환해서 새 배열이 필요하다면 map을 써야 합니다.

```
const names = ["Ada", "Linus", "Grace"];

names.forEach((name) => {
  console.log(`${name}님 환영합니다.`);
});

const wrong = names.forEach((name) => name.toUpperCase());
console.log(wrong); // undefined

const right = names.map((name) => name.toUpperCase());
console.log(right); // ["ADA", "LINUS", "GRACE"]
```

2.7 객체 접근: obj.name, obj["name"]

객체 속성을 읽는 방법은 점 표기법과 대괄호 표기법이 있습니다. 둘 다 같은 값을 읽을 수 있지만 쓰임이 다릅니다.

```
const user = {
  name: "Yoonji",
  "favorite-language": "JavaScript",
};

console.log(user.name);    // 점 표기법
console.log(user["name"]); // 대괄호 표기법

const key = "name";
console.log(user[key]);    // 변수에 들어 있는 key로 접근

console.log(user["favorite-language"]); // 특수문자 key는 대괄호 필요
```

접근법	언제 쓰나
obj.name	속성 이름이 코드에 고정되어 있고, 일반적인 변수 이름 규칙을 따를 때 사용합니다.
obj['name']	속성 이름에 특수문자가 있거나, 변수에 담긴 key로 동적으로 접근할 때 사용합니다.

2.8 객체 구조분해 할당

객체 구조분해 할당은 객체에서 필요한 속성을 꺼내 변수로 만드는 문법입니다.

왜 필요한가: 매번 user.name, user.age처럼 반복해서 쓰지 않고 필요한 값에 바로 이름을 붙일 수 있습니다.

- 속성 이름과 같은 변수명이 만들어집니다.
- 다른 변수명으로 받고 싶으면 name: userName처럼 씁니다.
- 기본값을 지정하면 속성이 없을 때 undefined 대신 기본값을 사용합니다.
- 함수 매개변수에서도 자주 사용합니다.

```
const user = {
  id: 1,
  name: "Yoonji",
  age: 21,
};

const { name, age } = user;
console.log(name, age); // "Yoonji", 21

const { name: userName, city = "Seoul" } = user;
console.log(userName); // "Yoonji"
console.log(city); // "Seoul"

function printUser({ name, age }) {
  console.log(`${name}: ${age}`);
}

printUser(user);
```

2.9 배열 구조분해 할당

배열 구조분해 할당은 배열의 위치를 기준으로 값을 꺼내 변수로 만드는 문법입니다.

- 첫 번째 변수는 배열의 0번 요소, 두 번째 변수는 1번 요소를 받습니다.
- 쉼표를 사용해 중간 요소를 건너뛸 수 있습니다.
- 기본값을 지정할 수 있습니다.
- React의 `useState`가 배열 구조분해 패턴을 사용합니다.

```
const colors = ["red", "green", "blue"];

const [first, second] = colors;
console.log(first); // "red"
console.log(second); // "green"

const [, , third] = colors;
console.log(third); // "blue"

const [value, setValue] = useState("");
```

2.10 spread 문법: ...

spread 문법은 배열이나 객체를 '펼쳐서' 다른 배열/객체/함수 호출 안에 넣는 문법입니다.

왜 필요한가: 원본을 직접 바꾸지 않고 복사본을 만들거나, 기존 값에 새 값을 추가하는 패턴에 유용합니다. React 상태 업데이트에서도 중요합니다.

- 배열에서는 요소들을 펼칩니다.
- 객체에서는 속성들을 펼칩니다.
- 겹겹질만 복사하는 얇은 복사입니다. 중첩 객체까지 자동으로 깊게 복사하지 않습니다.
- 뒤에 오는 속성이 앞의 같은 이름 속성을 덮어씁니다.

```
const numbers = [1, 2, 3];
const nextNumbers = [...numbers, 4];
console.log(nextNumbers); // [1, 2, 3, 4]

const user = { id: 1, name: "Yoonji" };
const updatedUser = { ...user, name: "YJ" };
```

```

console.log(updatedUser); // { id: 1, name: "YJ" }

function add(a, b, c) {
  return a + b + c;
}

console.log(add(...numbers)); // 6

```

2.11 rest 문법: ...

rest 문법도 ...을 쓰지만 의미는 spread와 반대입니다. 여러 값을 하나로 '모아서' 배열이나 객체로 받습니다.

- 함수 매개변수에서 남은 인자를 배열로 모을 수 있습니다.
- 배열 구조분해에서 앞에서 꺼낸 값을 제외한 나머지를 모을 수 있습니다.
- 객체 구조분해에서도 특정 속성을 제외한 나머지 속성을 모을 수 있습니다.
- spread는 펼치기, rest는 모으기입니다. 같은 기호지만 위치로 의미가 결정됩니다.

```

function sum(...numbers) {
  return numbers.reduce((total, number) => total + number, 0);
}

console.log(sum(1, 2, 3, 4)); // 10

const [first, ...others] = ["a", "b", "c"];
console.log(first); // "a"
console.log(others); // ["b", "c"]

const user = { id: 1, name: "Yoonji", age: 21 };
const { id, ...profile } = user;
console.log(profile); // { name: "Yoonji", age: 21 }

```

spread와 rest 구분

값을 넣는 자리에서 ...을 쓰면 보통 spread입니다. 값을 받는 자리, 즉 함수 매개변수나 구조분해 할당 왼쪽에서 ...을 쓰면 보통 rest입니다.

3. 비동기 처리

비동기 처리는 React에서 FastAPI 같은 서버를 호출할 때 반드시 이해해야 하는 부분입니다. 서버 응답은 즉시 오지 않을 수 있으므로, JavaScript는 기다리는 동안 화면을 멈추지 않고 나중에 도착한 결과를 처리하는 방식을 사용합니다.

3.1 동기 / 비동기

동기 코드는 위에서 아래로 한 줄씩 끝난 뒤 다음 줄로 넘어갑니다. 비동기 코드는 오래 걸리는 작업을 시작해 두고, 결과가 준비되면 나중에 이어서 처리합니다.

- 동기 작업은 순서를 예측하기 쉽지만 오래 걸리는 작업이 있으면 전체 흐름이 막힙니다.
- 비동기 작업은 네트워크 요청, 파일 읽기, 타이머처럼 시간이 걸리는 작업에 적합합니다.
- `fetch` 같은 서버 요청은 비동기입니다.
- 비동기를 제대로 처리하지 않으면 아직 도착하지 않은 값을 바로 쓰려고 해서 오류가 생깁니다.

```
console.log("1. 시작");

setTimeout(() => {
  console.log("2. 나중에 실행");
}, 1000);

console.log("3. 끝");

// 출력 순서:
// 1. 시작
// 3. 끝
// 2. 나중에 실행
```

3.2 Promise

Promise는 '미래에 성공하거나 실패할 수 있는 작업의 결과'를 표현하는 객체입니다.

- `pending`은 아직 결과가 없는 대기 상태입니다.
- `fulfilled`는 작업이 성공한 상태입니다.
- `rejected`는 작업이 실패한 상태입니다.
- `then`은 성공 결과를 처리하고, `catch`는 실패를 처리하고, `finally`는 성공/실패와 관계없이 마지막에 실행됩니다.
- 요즘은 Promise를 직접 `then`으로 길게 연결하기보다 `async/await`으로 읽기 쉽게 쓰는 경우가 많습니다.

```
fetch("/api/users")
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.error("요청 실패:", error);
  })
  .finally(() => {
    console.log("요청 종료");
  });
```

3.3 async / await

async/await은 Promise 기반 비동기 코드를 동기 코드처럼 위에서 아래로 읽히게 해 주는 문법입니다.

- async가 붙은 함수는 항상 Promise를 반환합니다.
- await은 Promise가 처리될 때까지 해당 async 함수 안에서 다음 줄 실행을 기다립니다.
- await은 보통 async 함수 안에서 사용합니다.
- 여러 요청을 순서대로 기다리면 느릴 수 있습니다. 동시에 시작해도 되는 작업은 Promise.all을 고려합니다.

```
async function loadUsers() {
  const response = await fetch("/api/users");
  const users = await response.json();
  return users;
}

async function main() {
  const users = await loadUsers();
  console.log(users);
}
```

실행 흐름

await은 전체 브라우저를 멈추는 것이 아니라, 그 async 함수의 다음 줄로 넘어가는 것을 잠시 미룹니다. 그래서 화면은 계속 반응할 수 있고, 결과가 도착하면 이어서 실행됩니다.

3.4 fetch

fetch는 브라우저에서 HTTP 요청을 보내는 기본 함수입니다. 서버에서 데이터를 가져오거나 서버에 데이터를 보낼 때 사용합니다.

- fetch(url)는 Promise<Response>를 반환합니다.
- 응답 본문을 JSON으로 읽으려면 await response.json()을 한 번 더 호출해야 합니다.
- 404나 500 같은 HTTP 오류 상태여도 fetch 자체가 자동으로 throw하지 않습니다. response.ok를 직접 확인하는 습관이 좋습니다.
- POST/PUT/PATCH 요청에서는 method, headers, body 옵션을 자주 사용합니다.

```
async function getTodos() {
  const response = await fetch("http://localhost:8000/todos");

  if (!response.ok) {
    throw new Error(`HTTP 오류: ${response.status}`);
  }

  const data = await response.json();
  return data;
}

async function createTodo(title) {
  const response = await fetch("http://localhost:8000/todos", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ title }),
  });
}
```

```

if (!response.ok) {
  throw new Error('생성 실패: ${response.status}');
}

return await response.json();
}

```

3.5 try / catch

try/catch는 실행 중 발생한 에러를 잡아서 프로그램이 갑자기 멈추지 않도록 처리하는 문법입니다.

- try 블록 안에 실패할 수 있는 코드를 넣습니다.
- catch 블록에서는 에러를 사용자에게 알리거나 로그로 남깁니다.
- throw new Error(...)로 직접 에러를 만들 수 있습니다.
- 비동기 코드에서는 await와 함께 try/catch를 쓰면 실패 처리가 읽기 쉽습니다.

```

async function loadTodos() {
  try {
    const response = await fetch("http://localhost:8000/todos");

    if (!response.ok) {
      throw new Error('HTTP 오류: ${response.status}');
    }

    const todos = await response.json();
    return todos;
  } catch (error) {
    console.error(error);
    return [];
  }
}

```

3.6 JSON

JSON은 JavaScript Object Notation의 약자이며, 서버와 클라이언트가 데이터를 주고받을 때 자주 쓰는 텍스트 형식입니다.

- JSON의 key는 반드시 큰따옴표로 감쌉니다.
- 문자열 값도 큰따옴표를 사용합니다.
- 함수, undefined, 주석은 JSON 데이터에 넣을 수 없습니다.
- JSON.stringify는 JavaScript 값을 JSON 문자열로 바꿉니다.
- JSON.parse는 JSON 문자열을 JavaScript 값으로 바꿉니다.
- fetch 응답의 response.json()은 응답 본문을 JavaScript 객체/배열로 파싱합니다.

```

const user = {
  id: 1,
  name: "Yoonji",
};

const jsonText = JSON.stringify(user);
console.log(jsonText); // {"id":1,"name":"Yoonji"}

const parsed = JSON.parse(jsonText);

```

```

console.log(parsed.name); // "Yoonji"

// 서버로 보낼 때
fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(user),
});

```

3.7 HTTP 요청/응답

HTTP는 브라우저와 서버가 요청(request)과 응답(response)을 주고받는 규칙입니다. React가 FastAPI를 호출한다는 것은 React 코드가 HTTP 요청을 보내고, FastAPI가 HTTP 응답을 돌려준다는 뜻입니다.

- 요청은 보통 method, URL, headers, body로 구성됩니다.
- GET은 데이터를 조회할 때, POST는 새 데이터를 만들 때, PUT/PATCH는 수정할 때, DELETE는 삭제할 때 주로 씁니다.
- 응답은 status code, headers, body로 구성됩니다.
- 200번대는 성공, 400번대는 클라이언트 요청 문제, 500번대는 서버 문제로 이해하면 됩니다.
- JSON을 주고받을 때는 Content-Type: application/json 헤더가 중요합니다.

예시	의미
GET /todos	할 일 목록 조회
POST /todos	새 할 일 생성. 보통 body에 JSON 데이터를 담습니다.
PATCH /todos/1	1번 할 일 일부 수정
DELETE /todos/1	1번 할 일 삭제
200 OK	요청 성공
201 Created	생성 성공
400 Bad Request	요청 형식이나 값이 잘못됨
401 Unauthorized	로그인이 필요하거나 인증 실패
404 Not Found	해당 URL 또는 데이터가 없음
500 Internal Server Error	서버 내부 오류

3.8 React에서 FastAPI 호출 예시

아래 예시는 React 컴포넌트가 처음 화면에 나타날 때 FastAPI 서버에서 할 일 목록을 가져오고, 실패하면 에러 메시지를 보여 주는 기본 패턴입니다.

```

import { useEffect, useState } from "react";

const API_BASE_URL = "http://localhost:8000";

export default function TodoList() {
  const [todos, setTodos] = useState([]);

```



```

const [isLoading, setIsLoading] = useState(true);
const [errorMessage, setErrorMessage] = useState("");

useEffect(() => {
  async function loadTodos() {
    try {
      const response = await fetch(`${API_BASE_URL}/todos`);

      if (!response.ok) {
        throw new Error(`HTTP 오류: ${response.status}`);
      }

      const data = await response.json();
      setTodos(data);
    } catch (error) {
      setErrorMessage("할 일 목록을 불러오지 못했습니다.");
    } finally {
      setIsLoading(false);
    }
  }

  loadTodos();
}, []);

if (isLoading) return <p>불러오는 중...</p>;
if (errorMessage) return <p>{errorMessage}</p>;

return (
  <ul>
    {todos.map((todo) => (
      <li key={todo.id}>{todo.title}</li>
    ))}
  </ul>
);
}

```

코드 부분	역할
useEffect	컴포넌트가 렌더링된 뒤 서버 요청을 시작합니다.
useState	서버에서 받은 데이터, 로딩 상태, 에러 메시지를 화면 상태로 저장합니다.
fetch	FastAPI 엔드포인트로 HTTP 요청을 보냅니다.
await response.json()	응답 JSON을 JavaScript 배열/객체로 바꿉니다.
try/catch/finally	성공, 실패, 마무리 처리를 분리합니다.
todos.map	서버에서 받은 배열을 화면의 li 목록으로 변환합니다.

CORS 참고

React 개발 서버와 FastAPI 서버의 주소나 포트가 다르면 브라우저가 CORS 정책을 검사합니다. JavaScript 문법 문제는 아니지만, 실제 연결에서 막히면 FastAPI 쪽 CORS 설정도 확인해야 합니다.

4. 모듈

프로젝트가 커지면 모든 코드를 한 파일에 둘 수 없습니다. 모듈 문법은 필요한 값, 함수, 컴포넌트를 파일 밖으로 내보내고 다른 파일에서 가져오게 해 줍니다.

4.1 export

export는 현재 파일 안의 변수, 함수, 클래스를 다른 파일에서 사용할 수 있도록 내보내는 문법입니다.

- 이름 붙은 export는 한 파일에서 여러 개 사용할 수 있습니다.
- 선언 앞에 export를 붙이거나, 파일 아래쪽에서 한 번에 export할 수 있습니다.

```
// math.js
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

export { subtract };
```

4.2 import

import는 다른 파일이나 패키지에서 export된 값을 가져오는 문법입니다.

- 상대 경로 파일은 ./ 또는 ../로 시작합니다.
- 패키지는 react처럼 패키지 이름으로 가져옵니다.
- named export는 중괄호 안에 정확한 이름을 써서 가져옵니다.
- default export는 중괄호 없이 가져옵니다.

```
// app.js
import { PI, add, subtract } from "./math.js";
import React from "react";

console.log(PI);
console.log(add(2, 3));
console.log(subtract(5, 2));
```

4.3 default export

default export는 파일의 대표 값을 하나 내보내는 문법입니다. 한 파일에는 default export를 하나만 둘 수 있습니다.

- 가져올 때 중괄호를 쓰지 않습니다.
- 가져오는 쪽에서 원하는 이름을 붙일 수 있습니다.
- React 컴포넌트 파일에서 자주 사용합니다.

```
// Button.jsx
export default function Button({ label }) {
  return <button>{label}</button>;
}
```

```
// App.jsx
import Button from './Button.jsx';
import PrimaryButton from './Button.jsx'; // 이름을 다르게 붙여도 가능
```

4.4 named export

named export는 이름이 있는 여러 값을 내보내는 문법입니다. 가져올 때 같은 이름을 중괄호 안에 써야 합니다.

- 한 파일에서 여러 개를 내보내기 좋습니다.
- 가져올 때 이름이 정확히 일치해야 합니다.
- as를 사용하면 가져오는 쪽에서 별칭을 붙일 수 있습니다.
- 유틸 함수, 상수, API 함수 묶음에 자주 사용합니다.

```
// api.js
export async function getTodos() {
  const response = await fetch("/api/todos");
  return await response.json();
}

export async function createTodo(title) {
  const response = await fetch("/api/todos", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title })
  });

  return await response.json();
}

// TodoPage.jsx
import { getTodos, createTodo as addTodo } from './api.js';
```

문법	핵심 차이
default export	파일의 대표값 하나. import Button from './Button.jsx'처럼 중괄호 없이 가져옵니다.
named export	여러 값을 이름으로 내보냄. import { getTodos } from './api.js'처럼 중괄호로 가져옵니다.
export	현재 파일의 값을 밖으로 공개합니다.
import	다른 파일의 공개된 값을 현재 파일로 가져옵니다.

5. 통합 예제: API 모듈을 분리한 React 코드

아래 예시는 지금까지 배운 개념을 함께 사용합니다. api.js 파일은 서버 통신만 담당하고, TodoPage.jsx는 화면 상태와 렌더링만 담당합니다.

5.1 api.js

```
const API_BASE_URL = "http://localhost:8000";

async function requestJson(path, options = {}) {
  const response = await fetch(`${API_BASE_URL}${path}`, {
    headers: {
      "Content-Type": "application/json",
      ...options.headers,
    },
    ...options,
  });

  if (!response.ok) {
    throw new Error(`HTTP 오류: ${response.status}`);
  }

  return await response.json();
}

export function getTodos() {
  return requestJson("/todos");
}

export function createTodo(title) {
  return requestJson("/todos", {
    method: "POST",
    body: JSON.stringify({ title }),
  });
}
```

5.2 TodoPage.jsx

```
import { useEffect, useState } from "react";
import { createTodo, getTodos } from "../api.js";

export default function TodoPage() {
  const [todos, setTodos] = useState([]);
  const [title, setTitle] = useState("");
  const [errorMessage, setErrorMessage] = useState("");

  useEffect(() => {
    async function load() {
      try {
        const data = await getTodos();
        setTodos(data);
      } catch (error) {
        setErrorMessage("목록을 불러오지 못했습니다.");
      }
    }

    load();
  }, []);

  async function handleSubmit(event) {
    event.preventDefault();

    try {
      const newTodo = await createTodo(title);
      setTodos((prevTodos) => [...prevTodos, newTodo]);
      setTitle("");
    } catch (error) {
      setErrorMessage("저장하지 못했습니다.");
    }
  }

  return (
    <section>
      <form onSubmit={handleSubmit}>
        <input value={title} onChange={(e) => setTitle(e.target.value)} />
        <button type="submit">추가</button>
      </form>

      {errorMessage && <p>{errorMessage}</p>}

      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>{todo.title}</li>
        ))}
      </ul>
    </section>
  );
}
```

코드	사용된 개념
const API_BASE_URL	서버 주소를 변수로 저장합니다.
async function requestJson	fetch, response.ok 검사, JSON 파싱을 함수로 묶습니다.
...options.headers	기본 헤더와 추가 헤더를 spread로 합칩니다.

코드	사용된 개념
<code>export function getTodos</code>	다른 파일에서 쓸 수 있도록 API 함수를 named export합니다.
<code>import { createTodo, getTodos }</code>	named export된 함수를 가져옵니다.
<code>const [todos, setTodos]</code>	배열 구조분해로 React 상태값과 setter를 받습니다.
<code>try/catch</code>	네트워크 실패나 HTTP 오류를 잡습니다.
<code>setTodos((prevTodos) => [...prevTodos, newTodo])</code>	기존 배열을 직접 바꾸지 않고 spread로 새 배열을 만듭니다.
<code>todos.map</code>	배열 데이터를 화면 요소로 변환합니다.

6. 자주 헛갈리는 포인트

질문	답
const 배열에 push 가능?	가능합니다. const 는 변수 재할당을 막을 뿐 배열 내부 변경까지 막지 않습니다. React 상태에서는 직접 push 보다 새 배열을 만드는 방식이 권장됩니다.
map 과 forEach 차이	map 은 새 배열을 반환하고, forEach 는 반환값이 undefined 입니다. 화면 목록이나 변환 결과가 필요하다면 map 을 씁니다.
find 결과가 없으면?	undefined 입니다. user.name 처럼 바로 접근하면 에러가 날 수 있으니 if (user) 로 확인합니다.
return 을 빼면?	함수 반환값은 undefined 가 됩니다. 화살표 함수에서 중괄호를 쓰면 return 을 직접 써야 합니다.
await response.json() 을 왜 또 하나?	fetch 는 먼저 Response 객체를 주고, 본문 파싱은 별도 비동기 작업입니다.
HTTP 404 에서 catch 가 바로 실행?	fetch 는 네트워크 자체 실패가 아니면 404만으로 자동 reject 하지 않습니다. response.ok 를 확인하고 직접 throw 하는 패턴이 안전합니다.
JSON 과 객체는 같은가?	비슷하게 생겼지만 JSON 은 텍스트 형식이고, 객체는 JavaScript 메모리 안의 값입니다. 변환에는 JSON.stringify/parse 가 필요합니다.
default 와 named import 혼동	default 는 중괄호 없이, named 는 중괄호로 가져옵니다.

7. 전체 개념 체크리스트

아래 항목을 스스로 설명할 수 있으면 이 문서의 기본 범위를 잘 이해한 것입니다.

개념	이해 기준
let, const	변수 선언과 재할당 가능 여부를 설명할 수 있다.
string, number, boolean	문자열, 숫자, 참/거짓 값을 구분할 수 있다.
null, undefined	의도적으로 비운 값과 아직 없는 값을 구분할 수 있다.
Array	인덱스와 length 를 사용해 배열을 읽을 수 있다.
Object	key-value 구조로 대상을 표현할 수 있다.
if, else, switch	조건에 따라 실행 흐름을 나눌 수 있다.
for, for...of, map	반복 목적에 맞는 문법을 선택할 수 있다.
함수 선언	function 으로 작업을 묶고 호출할 수 있다.
화살표 함수	() => {} 형태의 콜백과 짧은 반환을 읽을 수 있다.
return	함수 결과와 함수 종료 시점을 이해할 수 있다.
템플릿 리터럴	백틱과 \${} 로 문자열을 만들 수 있다.

개념	이해 기준
map, filter, find, reduce, forEach	각 배열 메서드의 반환값과 사용 목적을 구분할 수 있다.
obj.name, obj['name']	고정 key와 동적 key 접근을 구분할 수 있다.
객체 구조분해	객체 속성을 변수로 꺼낼 수 있다.
배열 구조분해	배열 위치에 따라 값을 변수로 꺼낼 수 있다.
spread	배열/객체를 펼쳐 복사하거나 합칠 수 있다.
rest	여러 값을 하나로 모을 수 있다.
동기 / 비동기	즉시 실행과 나중에 완료되는 작업의 차이를 설명할 수 있다.
Promise	pending, fulfilled, rejected 상태를 이해할 수 있다.
async / await	Promise 결과를 읽기 쉬운 순서로 처리할 수 있다.
fetch	HTTP 요청을 보내고 응답 JSON을 읽을 수 있다.
try / catch	실패할 수 있는 코드를 안전하게 처리할 수 있다.
JSON	객체와 JSON 문자열을 변환할 수 있다.
HTTP 요청/응답	method, URL, headers, body, status의 역할을 설명할 수 있다.
import/export	파일 사이에서 값을 가져오고 내보낼 수 있다.
default export	파일의 대표값 하나를 중괄호 없이 가져올 수 있다.
named export	여러 값을 이름으로 내보내고 중괄호로 가져올 수 있다.

8. 마지막 요약

- 변수는 값을 이름으로 저장하고, 배열과 객체는 여러 값을 구조화합니다.
- 조건문과 반복문은 코드 실행 흐름을 제어합니다.
- 함수는 반복되는 작업에 이름을 붙이고, return은 결과를 함수 밖으로 돌려줍니다.
- map/filter/find/reduce/forEach는 배열 데이터를 읽고 바꾸는 핵심 도구입니다.
- 구조분해, spread, rest는 배열과 객체를 간결하게 다루게 해 줍니다.
- Promise, async/await, fetch, try/catch는 서버 응답처럼 나중에 도착하는 값을 안전하게 처리합니다.
- JSON과 HTTP 요청/응답을 이해하면 React에서 FastAPI를 호출하는 코드가 훨씬 명확해집니다.
- import/export는 코드를 역할별 파일로 나누어 유지보수하기 쉽게 만듭니다.